



OmniLink CRM

Final Software Project Documentation

An Integrated Customer Relationship Management Platform for Retail Customer Intelligence, Loyalty, Campaigns, Consent, and Customer Service

Prepared by: Harshit Singh and Ankit Kumar

Team: Team Syndicate

Project Mentor: Mrs.Varsha Domb

Submission 24/07/2026

Date:

Technology: HTML, CSS, JavaScript, FastAPI, Python, SQLite



Declaration

We hereby declare that the project report titled “OmniLink CRM: An Integrated Customer Relationship Management Platform for Retail Customer Intelligence, Loyalty, Campaigns, Consent, and Customer Service” is the result of our own work carried out as part of our final project. This report documents the design, development, implementation, integration, and testing of the OmniLink CRM system. The information, diagrams, screenshots, technical descriptions, and results included in this report are based on the work completed by our team during the project period. We further declare that, to the best of our knowledge, the content presented in this report is original and has not been submitted elsewhere for any other academic or professional purpose. Any external references, tools, frameworks, libraries, or documentation used during the development of this project have been appropriately acknowledged in the report. This project was completed by Team Syndicate under the guidance and supervision of our project mentor.

Team Syndicate

Acknowledgements

We, Team Syndicate, would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our project, OmniLink CRM: An Integrated Customer Relationship Management Platform for Retail Customer Intelligence, Loyalty, Campaigns, Consent, and Customer Service.

First and foremost, we would like to extend our heartfelt thanks to our project mentor, Mrs. Varsha Domb, for her valuable guidance, constant encouragement, and continuous support during the entire project journey.

We are also sincerely thankful to Sharvari Natekar and Harshita Doshi for staying regularly in touch with us throughout the development process. Their timely feedback, suggestions, and reviews helped us remain aligned with the expected project requirements. Their inputs allowed us to identify areas of improvement, refine the application flow, improve the documentation quality, and make the project more structured, complete, and presentable. Their regular communication and support played an important role in helping us maintain consistency and progress during each phase of development.

Every phase of the project helped us learn, improve, and work together more effectively as a team. We are thankful to everyone who supported us throughout this project and helped us make OmniLink CRM a successful learning and development experience.

Abstract

OmniLink CRM is a full-stack Customer Relationship Management prototype developed to unify retail customer intelligence and operational workflows. The platform addresses the fragmentation of customer, loyalty, consent, campaign, and service data by using an enriched Customer 360 foundation connected to specialized business modules. The Dashboard presents live KPIs and charts; Customer 360 provides searchable customer profiles; Consent Management controls channel permissions and Do-Not-Contact status; Loyalty Management supports membership tiers, benefits, points, redemptions, and recommendation logic; Campaign Management supports audience selection, consent filtering, message personalization, simulated Discord delivery, and response tracking; Customer Service supports ticket operations, follow-ups, resolutions, SLA information, and event timelines; and the Admin and Audit modules provide operational control and traceability.

The frontend is implemented using HTML, CSS, and, JavaScript. FastAPI provides modular REST APIs and SQLite provides a portable prototype database. JWT authentication, role-based authorization, PBKDF2 password hashing, masked information, audit logs, caching, pagination, and SQL aggregation support security and performance. The completed platform demonstrates an integrated CRM workflow in which one customer foundation supports analytics, loyalty decisions, compliant engagement, service operations, and governance.

Keywords: Customer Relationship Management, Customer 360, FastAPI, SQLite, Loyalty, Consent, Campaign Management, Customer Service, JWT, Role-Based Access Control.

Contents

Declaration	i
Acknowledgements	ii
Abstract	iii
List of Acronyms	vii
1 Introduction and Executive Overview	1
1.1 Executive Summary	1
1.2 Business Context	1
1.3 Problem Statement	2
1.4 Proposed Business Solution	2
2 Objectives, Scope, and Stakeholders	3
2.1 Project Objectives	3
2.2 Functional Scope	3
2.3 Stakeholders and Roles	4
2.4 Prototype Boundaries	4
3 System Architecture and Technology	5
3.1 Logical Architecture	5
3.1.1 Frontend Layer	5
3.1.2 API and Security Layer	5
3.1.3 Service Layer	5
3.1.4 Database Layer	5
3.2 Technology Stack	6
3.3 Project Structure	7
4 Authentication and Access Control	9
4.1 Login Workflow	9
4.2 JWT and Permission Checks	9
4.3 Password Hashing	9
4.4 Forgot Password	10

5	Database Design and Data Model	11
5.1	Central Customer Foundation	11
5.2	Operational Tables	11
5.3	Snapshot and History Pattern	12
6	Business Modules and Workflows	13
6.1	Dashboard Module	13
6.1.1	Primary KPIs	13
6.2	Customer 360 Module	13
6.3	Consent Management	15
6.4	Loyalty Management	15
6.4.1	Tiers and Benefits	15
6.4.2	Point Earning and Redemption	15
6.4.3	Recommendation Logic	16
6.5	Campaign Management	17
6.6	Customer Service	17
6.7	Admin, Audit, and Reports	18
7	API Design and End-to-End Operation	20
7.1	Main API Groups	20
7.2	Standard Protected Request	20
7.3	Write Operation Pattern	20
7.4	Error Handling	21
8	Security, Performance, and Reliability	22
8.1	Security and Privacy Controls	22
8.2	Performance Controls	22
8.3	Reliability Controls	23
8.4	Current Limitations	23
9	Testing, Deployment, and Results	24
9.1	Testing Approach	24
9.2	Same-Wi-Fi Demonstration Deployment	24
9.3	Demonstration Sequence	24
9.4	Results	25

10 Conclusion and Future Work	26
10.1 Conclusion	26
10.2 Future Work	26
10.3 Final Project Contribution	26

List of Acronyms

Acronym	Meaning
CRM	Customer Relationship Management
JWT	JSON Web Token
RBAC	Role-Based Access Control
API	Application Programming Interface
KPI	Key Performance Indicator
CLV	Customer Lifetime Value
DNC	Do Not Contact
SLA	Service-Level Agreement
REST	Representational State Transfer
PBKDF2	Password-Based Key Derivation Function 2
UI	User Interface
ERD	Entity Relationship Diagram

Introduction and Executive Overview

1.1 Executive Summary

OmniLink CRM was developed to address a common retail-business problem: customer information is often distributed across separate systems for purchases, loyalty membership, marketing communication, consent, and support operations. This fragmentation makes it difficult for employees to understand the customer completely, identify business opportunities, communicate through permitted channels, and provide consistent service.

The platform provides a unified Customer 360 data foundation and connects that foundation with multiple operational modules. The Dashboard converts customer and operational data into Key Performance Indicators and charts. Customer 360 provides searchable profiles. Consent Management records communication permissions and Do-Not-Contact status. Loyalty Management supports membership tiers, benefits, points, redemptions, and recommendation logic. Campaign Management creates segmented audiences, validates consent, personalizes templates, demonstrates delivery through Discord, and records outcomes. Customer Service manages tickets, follow-ups, resolutions, timelines, SLA information, and workload. Admin and Audit provide operational control and traceability.

Main Project Outcomes

- Unified customer data and customer-level insights.
- Consent-aware customer engagement.
- Configurable loyalty and reward operations.
- Campaign targeting, personalization, simulation, and response tracking.
- Structured Customer Service ticket operations.
- Live Dashboard, Admin, and Audit visibility.
- Secure authentication and controlled module access.

1.2 Business Context

A retail organization interacts with customers through stores, websites, mobile applications, campaigns, membership programs, and service channels. Each interaction creates data such as identity, location, transactions, product preferences, consent choices, membership activity, campaign responses, and support history.

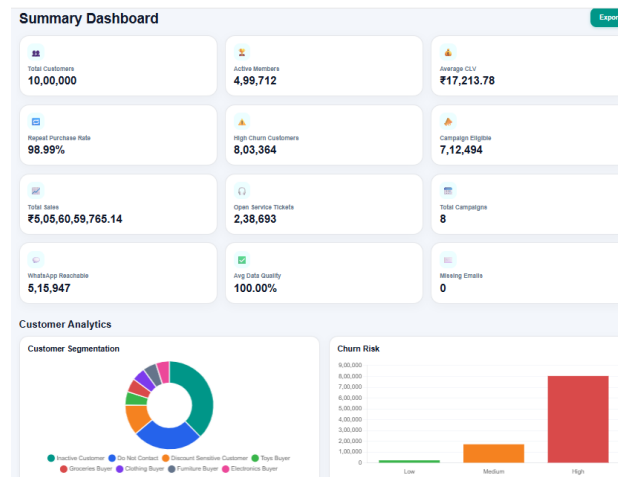


Figure 1.1: Executive overview presented by the OmniLink Dashboard.

Without a unified system, different teams may work with inconsistent customer information. Marketing may not know whether communication consent is active. Customer Service may not see loyalty status or churn risk. Management may not have a consolidated view of customer value, campaign performance, or service workload. Store teams may view more personal data than necessary. These issues produce operational inefficiency, poor customer experience, and privacy risk.

1.3 Problem Statement

The project addresses the following problems:

- Fragmented customer data across business functions.
- Absence of a consistent Customer 360 view.
- Campaign audience selection without a final consent check.
- Disconnected membership, reward, and upgrade operations.
- Limited customer and order context in service tickets.
- Static or manually prepared management analytics.
- Limited traceability of sensitive user actions.

1.4 Proposed Business Solution

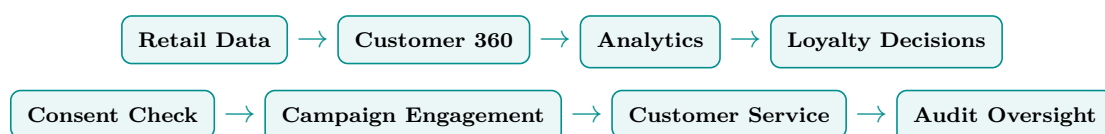


Figure 1.2: Compact business flow supported by OmniLink CRM.

Objectives, Scope, and Stakeholders

2.1 Project Objectives

1. Create a centralized customer intelligence platform.
2. Provide secure role-specific access to CRM modules.
3. Generate live KPIs and visual analytics from database data.
4. Support loyalty membership, reward points, and upgrade recommendations.
5. Ensure campaigns respect consent and Do-Not-Contact rules.
6. Provide a complete Customer Service ticket lifecycle.
7. Record important actions for audit and administrative review.
8. Demonstrate an integrated frontend-backend-database application.

2.2 Functional Scope

The system includes core business modules such as Dashboard Analytics, Customer 360, Consent Management, Loyalty Management, Campaign Management, and Customer Service. These modules support customer data visibility, consent-aware engagement, loyalty operations, campaign execution, service-ticket handling, and operational decision-making.

The prototype also includes supporting modules such as Reports, Audit Logs, Admin, and Settings. These modules provide reporting capability, user and operational monitoring, administrative control, traceability of important actions, and user profile management. In addition, the system includes shared frontend features such as common navigation, role-specific sidebar visibility, popup notifications, API caching, reusable helper functions, and same-network demonstration access for local deployment and testing.

2.3 Stakeholders and Roles

Table 2.1: Primary users and responsibilities.

Role	Accessible Pages / Modules
System Administrator	All pages
Marketing Team	Campaigns, Loyalty, and, Settings
Store Team	Customers, Loyalty, and, Settings
Management Team	Dashboard, Customers, Reports, and, Settings
Customer Service Team	Customers, Customer Service, and, Settings
IT Team	Admin, Audit Logs, and, Settings

2.4 Prototype Boundaries

SQLite is used instead of an enterprise database. Discord represents external campaign delivery. The password-reset email workflow may use a development fallback when external email authentication is unavailable. Some analytical fields originate from an enriched demonstration dataset rather than live retail systems.

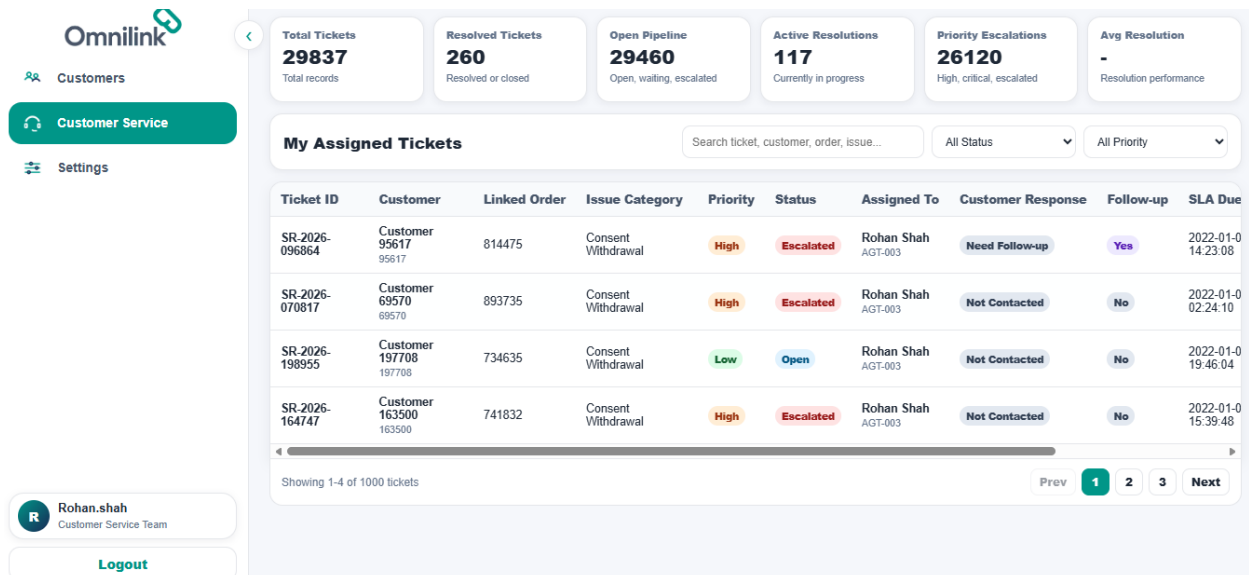


Figure 2.1: Role-specific module visibility in the shared navigation.

System Architecture and Technology

3.1 Logical Architecture

Table 3.1: Compact logical architecture.

Layer	Technology and responsibility
Presentation	Browser interface using HTML, CSS, JavaScript, and Chart.js.
Communication	REST/JSON over HTTP with JWT Bearer authentication.
Application	FastAPI routes, permission checks, validation, and business services.
Data Access	Reusable SQLite query helpers and parameterized SQL.
Persistence	SQLite CRM database containing customer and operational records.
External Demo	Discord webhook used for simulated campaign delivery.

3.1.1 Frontend Layer

The frontend renders forms, filters, tables, profiles, charts, popups, and operational workspaces. JavaScript sends authenticated API requests using the JWT Bearer token. Shared scripts provide navigation, page guards, notifications, formatting, and browser-side caching.

3.1.2 API and Security Layer

FastAPI organizes functionality into route groups. Each protected route reads the current user from the JWT and applies a permission check before running database or business logic.

3.1.3 Service Layer

Service files contain reusable business logic for password protection, loyalty recommendations, tier configuration, audit creation, consent decisions, caching, customer normalization, and external campaign delivery.

3.1.4 Database Layer

The SQLite client exposes reusable query helpers. The database stores customer snapshots, users, resets, campaigns, loyalty ledgers, tickets, timelines, and audit events.

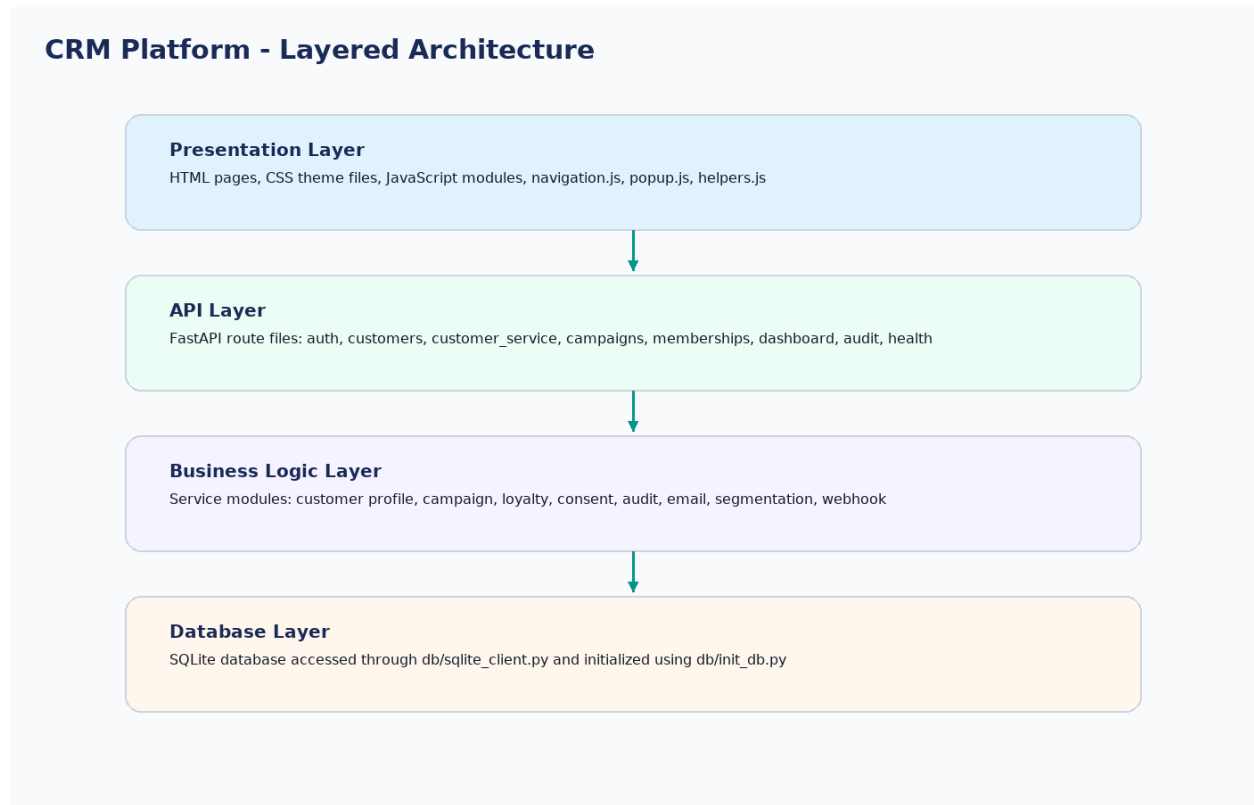


Figure 3.1: OmniLink CRM technical architecture.

3.2 Technology Stack

Table 3.2: Technology stack.

Area	Technology
Frontend	HTML, CSS, and, JavaScript
Visualization	Chart.js
Backend	Python, FastAPI
Application Server	Uvicorn
Database	SQLite
Authentication	JWT with role claims
Password Protection	PBKDF2-HMAC-SHA256 with random salt
API Testing	FastAPI Swagger UI
Campaign Demonstration	Discord webhooks
Documentation	Markdown and LaTeX/PDF

3.3 Project Structure

The implementation separates presentation, routing, configuration, database access, schemas, and reusable services. The compact folder structure below avoids implementation-level filename detail while still showing the system organization.

Table 3.3: Compact project folder structure.

Folder	Responsibility
frontend/pages/	CRM pages for Dashboard, Customer 360, Loyalty, Consent, Campaigns, Customer Service, Admin, Audit, Reports, and Settings.
frontend/css/	Shared and module-specific styling, responsive layouts, tables, cards, and popups.
frontend/js/	Rendering, API communication, navigation, caching, forms, filtering, charts, and sessions.
frontend/assets/	Logos, icons, and static visual resources.
omnilink-backend/ app/api/routes/	FastAPI route groups for authentication and CRM modules.
omnilink-backend/ app/core/	Configuration, JWT security, and permissions.
omnilink-backend/ app/db/	SQLite connection, initialization, and query helpers.
omnilink-backend/ app/schemas/	Pydantic validation models.
omnilink-backend/ app/services/	Reusable audit, password, cache, loyalty, consent, email, and integration logic.
omnilink-backend/	Database, configuration, migrations, dependencies, and documentation.

Listing 3.1: Folder-level backend structure.

```
omnilink-backend/
+-- app/
| +-- api/
| | +-- routes/
| +-- core/
| +-- db/
| +-- schemas/
| +-- services/
+-- database and setup scripts
+-- requirements and environment configuration
+-- project documentation
```

CRM Platform - Frontend and Backend Folder Structure

Frontend: Practice/

```
Practice/
├─ login.html, forgot-password.html
├─ popup.css, popup.js
├─ css/
│  └─ common.css, dashboard.css
│     └─ customer.css, customer-service.css
│        └─ campaign.css, loyalty.css
│           └─ Reports.css, settings.css
├─ js/
│  └─ navigation.js, helpers.js
│     └─ dashboard.js, customer.js
│        └─ customer-detail.js
│           └─ Customer Service.js
│              └─ campaign.js, loyalty.js
│                 └─ loyalty-profile.js
│                    └─ Reports.js, settings.js
│                       └─ audit.js, admin.js
├─ pages/
│  └─ dashboard.html, customer.html
│     └─ Customer Service.html
│        └─ campaign.html, loyalty.html
│           └─ loyalty-profile.html
│              └─ Reports.html, settings.html
│                 └─ audit.html, admin.html
```

Backend: app/

```
app/
├─ main.py
├─ api/routes/
│  └─ auth.py, customers.py
│     └─ customer_service.py
│        └─ campaigns.py, memberships.py
│           └─ consents.py, dashboard.py
│              └─ kpi.py, audit.py, health.py
├─ core/
│  └─ config.py
│     └─ security.py
├─ db/
│  └─ init_db.py
│     └─ sqlite_client.py
├─ schemas/
│  └─ schemas.py
├─ services/
│  └─ audit_service.py
│     └─ campaign_service.py
│        └─ consent_service.py
│           └─ customer_profile_service.py
│              └─ email_service.py
│                 └─ loyalty_service.py
│                    └─ segmentation_service.py
│                       └─ webhook_service.py
```

Figure 3.2: Project organization and registered API modules.

Authentication and Access Control

4.1 Login Workflow

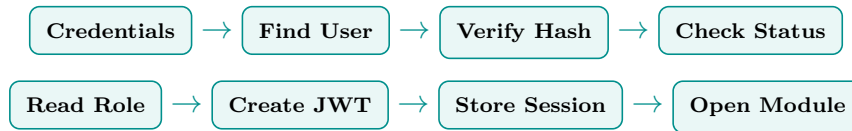


Figure 4.1: Compact authentication flow.

The backend, rather than the frontend role selector, is the authoritative source for the user role. Every protected API receives the JWT in the authorization header.

4.2 JWT and Permission Checks

The JWT contains the authenticated email in the `sub` claim, assigned role, and expiration. `get_current_user()` validates the signature and expiration. `require_permission()` checks whether the role may access the requested module.

4.3 Password Hashing

Passwords are stored using a random salt and PBKDF2-HMAC-SHA256. The login process derives a hash from the submitted password and stored salt, after which a constant-time comparison determines whether the password is correct. The system does not decrypt a password.

Listing 4.1: Simplified protected login logic.

```
user = fetch_one(
    "SELECT email, password, role, status "
    "FROM users WHERE lower(email) = ?",
    (email,),
)

if not user or not verify_password(password, user["password"]):
    raise HTTPException(status_code=401,
                        detail="Invalid email or password")

if user["status"].lower() != "active":
    raise HTTPException(status_code=403,
                        detail="User is inactive")
```

```
token = create_access_token(email, user["role"])
```

4.4 Forgot Password

The reset process generates a six-digit code, stores only a derived code hash with an expiration timestamp, and marks the record as used after a successful reset. The replacement password is hashed before being written to the `users` table.

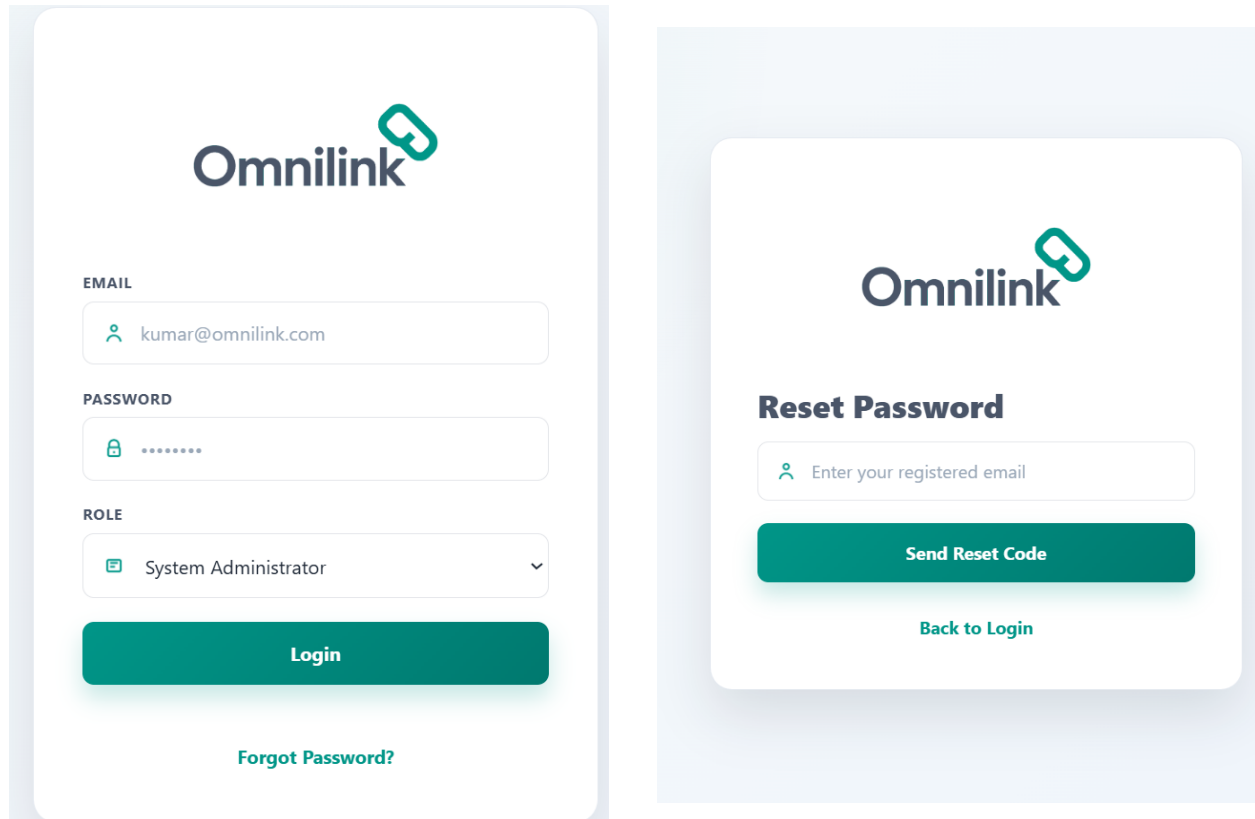


Figure 4.2: Login and password-recovery interfaces.

Database Design and Data Model

5.1 Central Customer Foundation

`customers_360` is the central enriched customer snapshot. It contains identity, masked and unmasked contact fields, geography, purchase behavior, product context, store and channel information, digital engagement, consent, membership, points, customer segments, churn, CLV, campaign eligibility, and data-quality indicators.

One row represents one enriched customer record. Different groups of columns are reused by Dashboard, Customer 360, Loyalty, Campaigns, Consent, and Customer Service.

5.2 Operational Tables

Table 5.1: Principal database tables.

Table	Purpose
<code>users</code>	Employee login identity, password hash, assigned role, and status.
<code>password_reset_codes</code>	Hashed reset code, expiry time, used state, and request timestamp.
<code>customers_360</code>	Central enriched customer snapshot used across CRM modules.
<code>consents</code>	Customer channel permissions and consent history where separate records are maintained.
<code>loyalty_tier_rules</code>	Tier order, points multiplier, spend unit, next tier, and recommendation threshold.
<code>loyalty_benefits</code>	Active benefits assigned to each membership tier.
<code>loyalty_point_transactions</code>	Append-only points earning and redemption ledger.
<code>loyalty_redemptions</code>	Business record of each completed or failed redemption.
<code>loyalty_tier_history</code>	Membership purchase and movement history.
<code>crm_campaigns</code>	Campaign definition, target information, status, and counters.
<code>crm_campaign_templates</code>	Reusable personalized campaign templates.
<code>crm_campaign_runs</code>	Individual campaign execution or simulation run.

Table	Purpose
<code>crm_campaign_message_logs</code>	Customer-level send, rendered content, response, error, and revenue details.
<code>customer_service_agents</code>	Agent identity, role, email, and availability.
<code>customer_service_tickets</code>	Current ticket snapshot and customer/order context.
<code>customer_service_timeline</code>	Append-only ticket lifecycle history.
<code>audit_logs</code>	User, action, object, status, details, and timestamp for traceability.

5.3 Snapshot and History Pattern

- Current points are stored in `customers_360`; point movement is stored in `loyalty_point_transaction`.
- Current ticket state is stored in `customer_service_tickets`; event history is stored in `customer_service_timeline`.
- Current membership tier is stored in `customers_360`; changes are stored in `loyalty_tier_history`.
- Campaign totals are stored in `crm_campaigns`; customer-level details are stored in `crm_campaign_message_logs`.

Business Modules and Workflows

6.1 Dashboard Module

The Dashboard is the primary executive analytics screen. The frontend calls `GET /dashboard/summary`; the backend aggregates data from Customer 360, campaigns, campaign message logs, service tickets, and loyalty records.

6.1.1 Primary KPIs

- **Total Customers:** Number of rows in `customers_360`.
- **Active Members:** Customers with an active purchased tier.
- **Member Percentage:** Active members divided by total customers, multiplied by 100.
- **Average CLV:** Average of `estimated_clv`.
- **Repeat Purchase Rate:** Repeat purchasers divided by total customers.
- **High-Churn Customers:** Customers classified with High churn risk.
- **Campaign Eligible:** Customers with at least one permitted campaign channel.
- **Total Sales:** Sum of `total_sales`.
- **Open Service Tickets:** Tickets not Resolved or Closed.
- **SLA Compliance:** Compliant tickets divided by applicable tickets.

Charts show customer segments, churn, tiers, cities, consent reachability, campaign status, campaign engagement, ticket status, ticket trend, SLA distribution, and data quality. The response is temporarily cached to avoid repeated aggregation.

6.2 Customer 360 Module

Customer 360 allows an authorized employee to search, inspect, and understand a customer from one interface. The list displays customer identity, tier, sales, estimated CLV, segment, churn risk, consent status, and a View action.

The profile combines identity, geography, masked contact information, purchase behavior, membership, points, expiry, preferred category, segment, potential-member score, CLV, churn risk, consent, and related operational history. Role-aware masking can restrict personal information, and views create audit events.

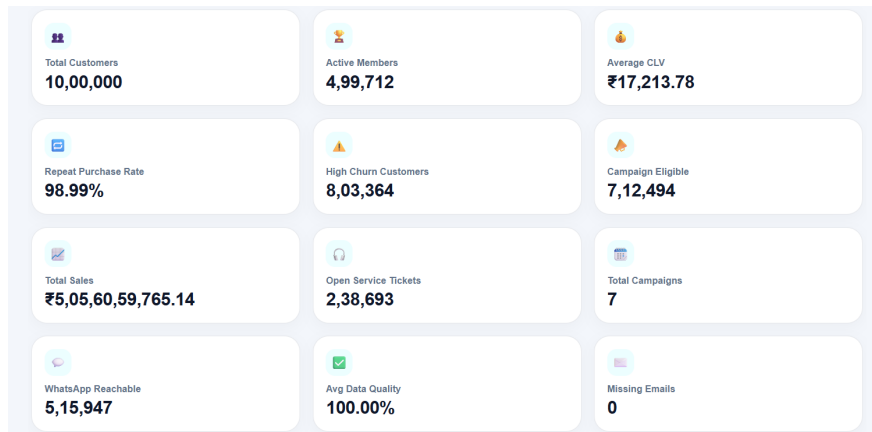


Figure 6.1: Live KPI cards, charts, and operational tables on the Dashboard.

Omnilink CRM interface showing a searchable list of customers. The left sidebar has navigation links: Customers, Customer Service, and Settings. The main area has a search bar and a table of customer records.

Customer	Tier	Spend	Segment	Consent	Action
Harshit Test 1 - 9000000001	Non-member	₹563.16 CLV: ₹16,005.96	Electronics Buyer Risk: Medium	Allowed: WhatsApp, Email	View
Customer 2 2 - 9000000002	Non-member	₹7,554.57 CLV: ₹217.94	Inactive Customer Risk: High	Allowed: WhatsApp	View
Customer 3 3 - 9000000003	Non-member	₹7,554.14 CLV: ₹5,437	Discount Sensitive Customer Risk: Low	Allowed: WhatsApp, SMS, Email	View
Customer 4 4 - 9000000004	Non-member	₹8,125.92 CLV: ₹426.33	Discount Sensitive Customer Risk: High	Allowed: WhatsApp	View
Customer 5 5 - 9000000005	Platinum	₹114.32 CLV: ₹26,895.36	Inactive Customer Risk: High	Allowed: WhatsApp, App	View
Customer 6 6 - 9000000006	Gold	₹3,372.17 CLV: ₹3,660.23	Do Not Contact Risk: High	Do Not Contact	View
Customer 7 7 - 9000000007	Non-member	₹1,322.64 CLV: ₹3,858.62	Do Not Contact Risk: High	Do Not Contact	View
Customer 8 8 - 9000000008	Platinum	₹1,716.65 CLV: ₹1,349.08	Groceries Buyer Risk: Medium	Allowed: WhatsApp, SMS, App	View

Figure 6.2: Searchable Customer 360 list.

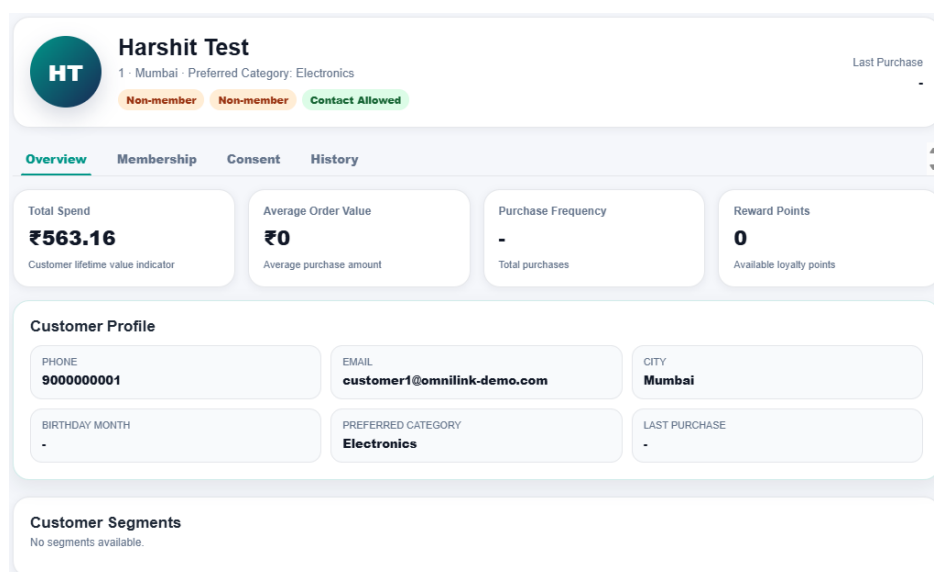


Figure 6.3: Detailed customer profile with loyalty, consent, and behavior information.

6.3 Consent Management

Consent Management separates opportunity from communication permission. A customer may remain a valid loyalty or campaign recommendation even when one or more channels are not allowed.

Listing 6.1: Consent eligibility rule.

```
If do_not_contact is enabled:
    Block promotional communication.
Else:
    Use a channel only if its channel-specific consent is enabled.
```

Authorized updates can change channel consent, personalization consent, preferred channel, source, or withdrawal state. A withdrawal clears promotional channels, enables Do-Not-Contact, records the withdrawal date, and creates an audit event.

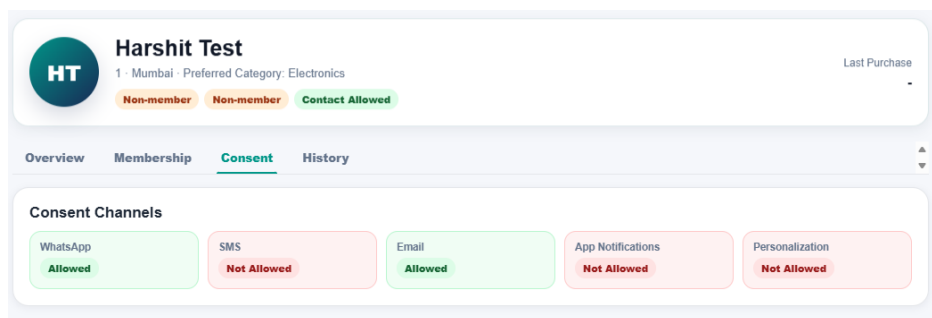


Figure 6.4: Channel-level permission and Do-Not-Contact controls.

6.4 Loyalty Management

The Loyalty module manages actual membership and rewards while producing internal recommendations for conversion, upgrade, and retention.

6.4.1 Tiers and Benefits

The configured tiers are Non-member, Silver, Gold, and Platinum. Customers may directly purchase a valid tier. Recommendation scores do not block a purchase. Rules and benefits are loaded from dedicated tables rather than being duplicated in frontend code.

6.4.2 Point Earning and Redemption

$$\text{Base Units} = \left\lfloor \frac{\text{Order Amount}}{\text{Spend Unit}} \right\rfloor \quad (6.1)$$

$$\text{Points Earned} = \text{Base Units} \times \text{Tier Multiplier} \quad (6.2)$$

The demonstration redemption conversion is 100 points for INR 10. Redemption validates minimum points, available balance, and the configured maximum share of order value.

6.4.3 Recommendation Logic

The upgrade score can use total sales, transaction frequency, estimated CLV, re-ency, repeat purchase, and digital engagement. Non-members are classified from `potential_member_score`. Members can be stable members, upgrade candidates, top-tier customers, or high-churn retention priorities.

The page uses pagination, limited previews, rule maps, TTL caches, and deferred calculations to avoid rescoring the full dataset during every page load.

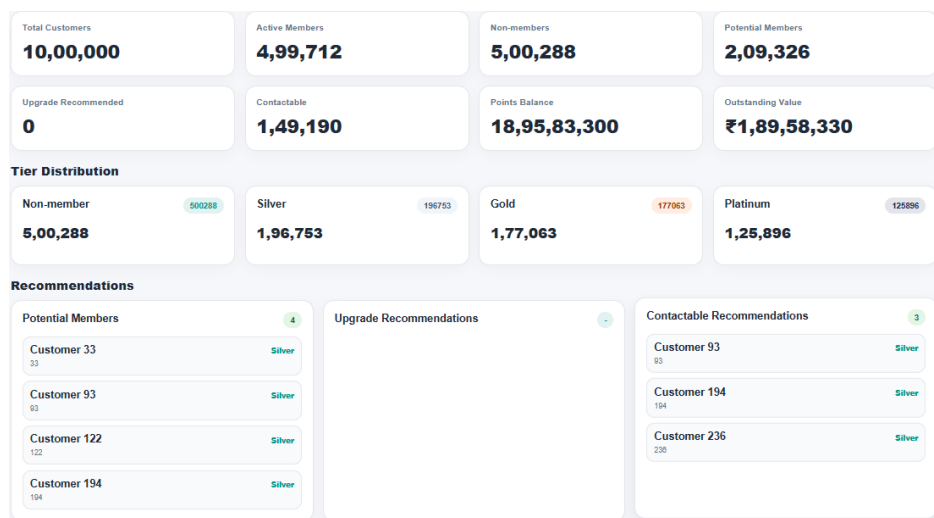


Figure 6.5: Loyalty summary, tier distribution, recommendations, benefits, and customer preview.

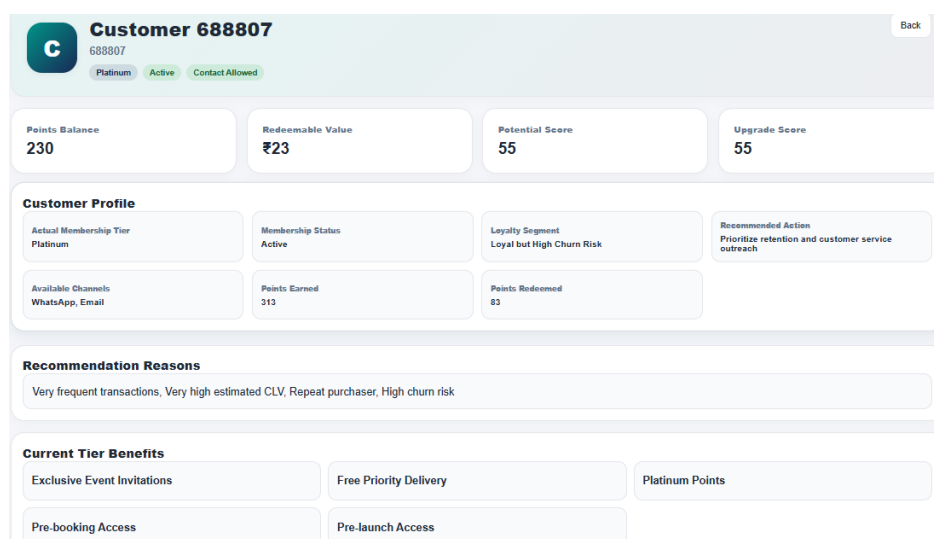


Figure 6.6: Customer-level loyalty profile, benefits, and transaction history.

6.5 Campaign Management

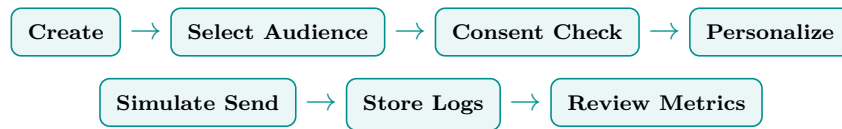


Figure 6.7: Compact campaign execution workflow.

Templates may contain customer name, tier, points balance, category, campaign name, and campaign ID placeholders. The backend selects the audience, validates consent, renders customer-specific content, maps the business channel to the demonstration webhook, sends the message, and records the result.

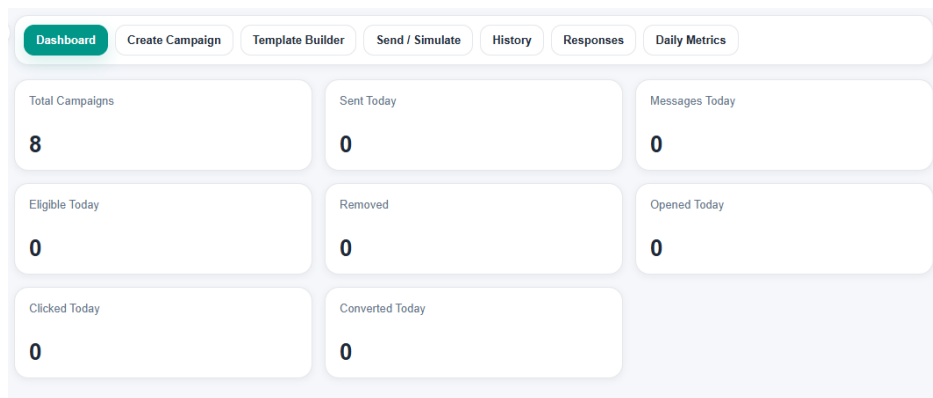


Figure 6.8: Campaign metrics, history, and operational controls.

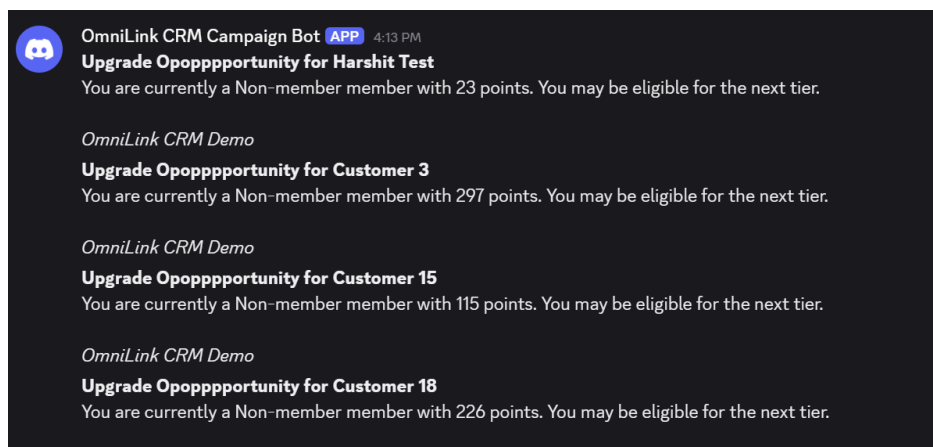


Figure 6.9: Campaign creation, audience filtering, and Discord demonstration delivery.

6.6 Customer Service

Customer Service supports a complete ticket lifecycle: creation, assignment, review, status updates, follow-ups, escalation, resolution, closure, timeline creation, and audit logging.

The current ticket row stores customer, issue, priority, status, assigned employee, SLA, order/product context, contact result, follow-up, internal notes, and resolution. The timeline stores immutable events explaining how the current state was reached.

Customer Service employees can be scoped to assigned tickets while administrators and managers can view broader queues and workload. Metrics summarize total, resolved, open, active, priority, and SLA-related tickets.

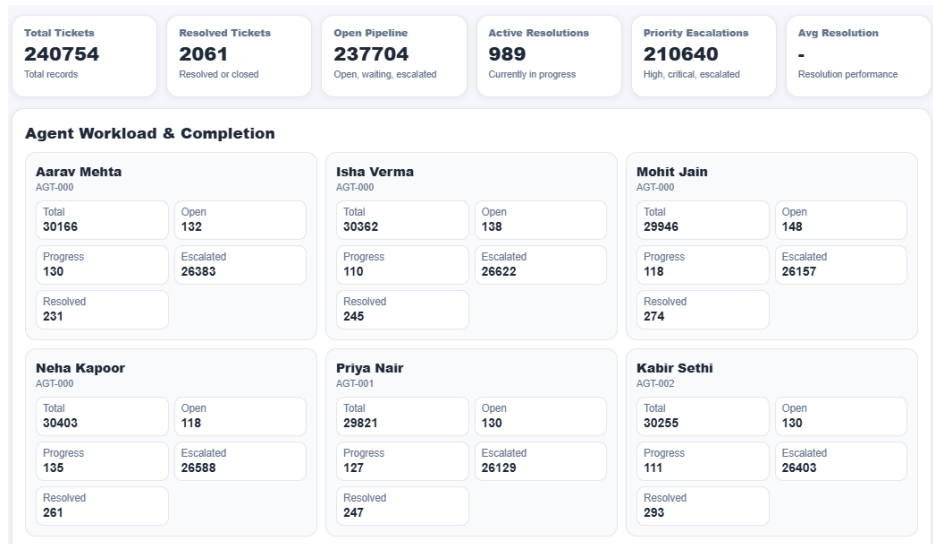


Figure 6.10: Customer Service metrics, filters, workload, and ticket queue.

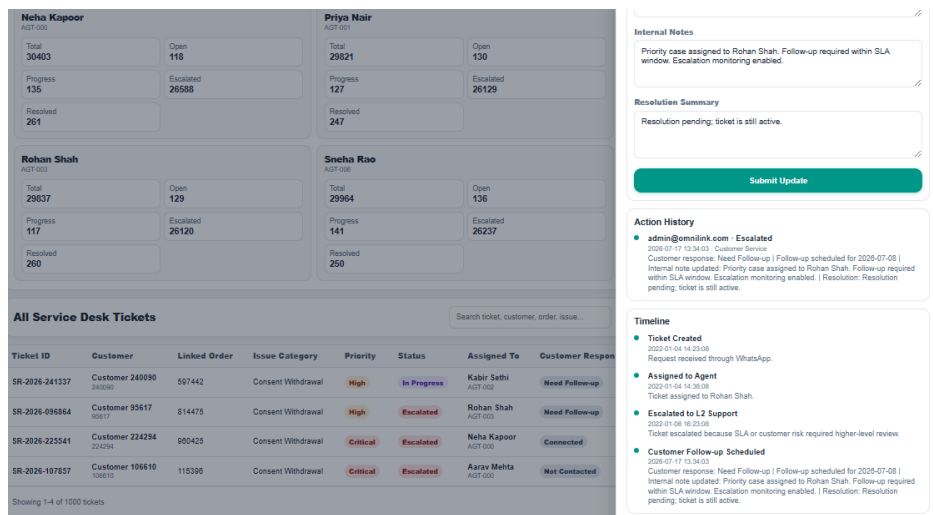


Figure 6.11: Ticket details, update form, action history, and timeline.

6.7 Admin, Audit, and Reports

The Admin module displays live summaries and controls for users, roles, agents, password reset requests, campaign status, data health, and CSV export. Passwords and reset-code hashes are never exposed to the frontend.

The Audit module provides summary cards, search, role/module/status/risk filters, pagination, and event details. Risk is derived from action and object type: security-sensitive changes and exports are High, operational updates are Medium, and routine views are Low.

The Reports module receives a Dashboard report draft from browser storage. Users can edit report text, switch layouts, preview the content, print to PDF, and export Word-compatible output.

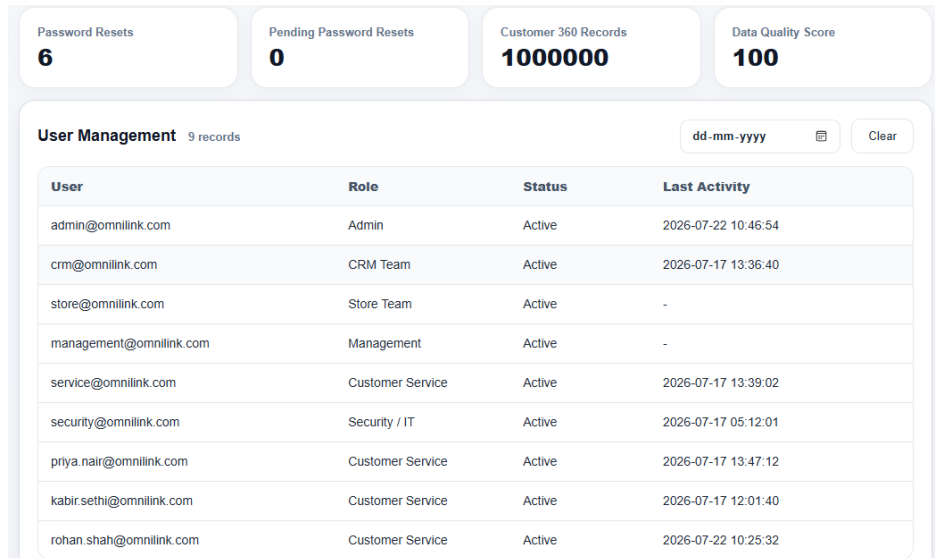


Figure 6.12: Administrative summaries and operational controls.

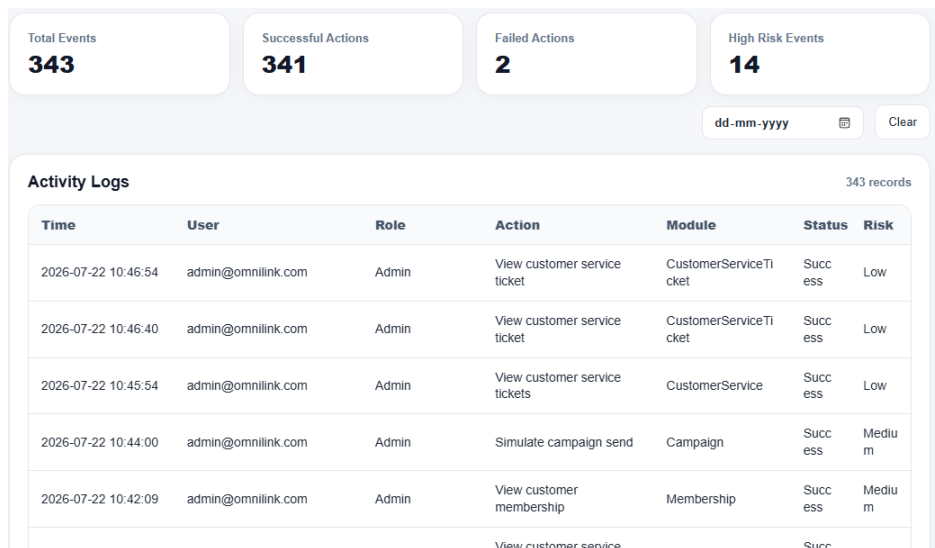


Figure 6.13: Searchable audit logs and derived risk classification.

API Design and End-to-End Operation

7.1 Main API Groups

Table 7.1: Principal REST API groups.

Prefix	Responsibility
/auth	Login, reset request, and password reset.
/dashboard	Executive summary, charts, and tables.
/customers	Customer list, profile, update, and timeline.
/consents	Consent list, detail, update, and withdrawal.
/memberships	Loyalty summary, members, rules, benefits, segments, purchase, earn, and redeem.
/campaigns	Campaign definitions, templates, preview, simulation, responses, and metrics.
/customer-service	Tickets, details, updates, metrics, and workload.
/admin	Users, agents, resets, campaigns, data health, and export.
/audit	Audit summary, filtered list, pagination, and detail.

7.2 Standard Protected Request

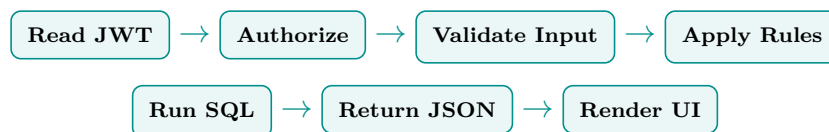


Figure 7.1: Compact protected-request lifecycle.

7.3 Write Operation Pattern

POST and PUT endpoints validate input and business rules, update a current-state table, append a ledger/history record when required, create an audit event, clear related cache entries, and return refreshed information.

Listing 7.1: Simplified protected route pattern.

```
@router.post("/{customer_id}/earn")
```

```
def earn_points(customer_id: str,
                payload: dict,
                user=Depends(get_current_user)):
    require_permission(user, "loyalty")
    customer = get_customer_row(customer_id)
    if not customer:
        raise HTTPException(404, "Customer not found")

    calculation = calculate_points_for_purchase(
        customer["membership_tier"],
        payload["order_amount"],
    )
    # Update customer balance, append ledger, create audit,
    # and invalidate summary/detail caches.
```

7.4 Error Handling

The API returns 400 for invalid input, 401 for invalid authentication, 403 for restricted access, 404 for missing records, and 409 for conflicts. The frontend converts backend detail messages into controlled popups or inline error states.

Authentication			^
POST	/auth/login	Login	▼
POST	/auth/forgot-password/request	Request Forgot Password	▼
POST	/auth/forgot-password/reset	Reset Forgot Password	▼
Customers			^
GET	/customers	Get Customers	🔒 ▼
GET	/customers/{customer_id}/profile	Get Customer Profile	🔒 ▼
PUT	/customers/{customer_id}/profile	Update Customer Profile	🔒 ▼
GET	/customers/{customer_id}/timeline	Get Customer Timeline	🔒 ▼
Dashboard			^
GET	/dashboard/summary	Get Dashboard Summary	🔒 ▼
Consents			^
GET	/consents	Get Consent List	🔒 ▼

Figure 7.2: REST API groups available through FastAPI Swagger UI.

Security, Performance, and Reliability

8.1 Security and Privacy Controls

- Salted PBKDF2 password hashing.
- JWT authentication with expiration.
- Backend role and module authorization.
- Active/inactive account status checks.
- Masked customer contact fields.
- Channel consent and Do-Not-Contact enforcement.
- Reset-code hashing, expiry, and one-time use.
- Audit logging of allowed, denied, and sensitive operations.
- No password or reset-code hash exposure through Admin APIs.
- Parameterized SQLite queries for dynamic values.

8.2 Performance Controls

- Direct SQL aggregation for summaries instead of per-customer calculation.
- Backend TTL cache for short-lived summary and configuration data.
- Browser session cache for reusable GET responses.
- Cache invalidation after database updates.
- Server-side pagination and result limits.
- Limited customer and recommendation previews for the demonstration.
- Map-based reuse of tier rules and benefits to prevent N+1 queries.
- SQLite indexes for frequently filtered or joined columns.

Listing 8.1: Simplified TTL-cache usage.

```
cached = cache.get("dashboard:summary")
if cached is not None:
    return cached

response = build_dashboard_summary()
```

```
cache.set("dashboard:summary", response,  
         ttl_seconds=60)  
return response
```

8.3 Reliability Controls

Validation rules protect ticket closure, follow-up scheduling, points redemption, campaign operations, and account changes. Timeline and ledger tables preserve history. Controlled empty states and partial-failure handling prevent an optional request from breaking an entire frontend page.

8.4 Current Limitations

- SQLite is not designed for high-concurrency enterprise deployment.
- In-memory cache is local to one server process.
- Discord is a demonstration transport rather than a production marketing provider.
- Some customer and analytical data is synthetic or enriched for demonstration.
- Production deployment requires HTTPS, centralized logs, stronger secrets management, background workers, and an enterprise database.

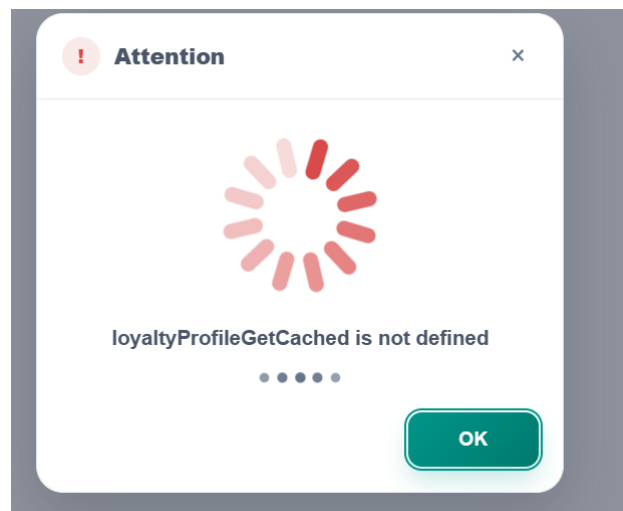


Figure 8.1: Secure request handling and short-lived cache strategy.

Testing, Deployment, and Results

9.1 Testing Approach

- Python syntax checks and import validation.
- Route testing using FastAPI Swagger UI.
- Database verification using SQLite queries and table inspection.
- Valid, invalid, inactive, and reset authentication scenarios.
- Permission testing across user roles.
- Frontend integration testing through browser developer tools.
- Membership purchase, earning, redemption, and history tests.
- Campaign creation, audience preview, simulation, and response tests.
- Ticket update, follow-up, resolution, and timeline tests.
- Cache hit, expiry, and invalidation tests.
- Same-Wi-Fi accessibility using a local IPv4 address.

9.2 Same-Wi-Fi Demonstration Deployment

The backend machine runs Uvicorn on all network interfaces:

Listing 9.1: Backend command for local-network access.

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

The frontend is served on port 3000 and configured to call the backend machine's local IPv4 address. CORS and the Windows Firewall must permit both the frontend origin and backend port.

9.3 Demonstration Sequence

1. Login and role-based navigation.
2. Dashboard KPIs and charts.
3. Customer 360 search and profile.
4. Loyalty recommendation and profile.

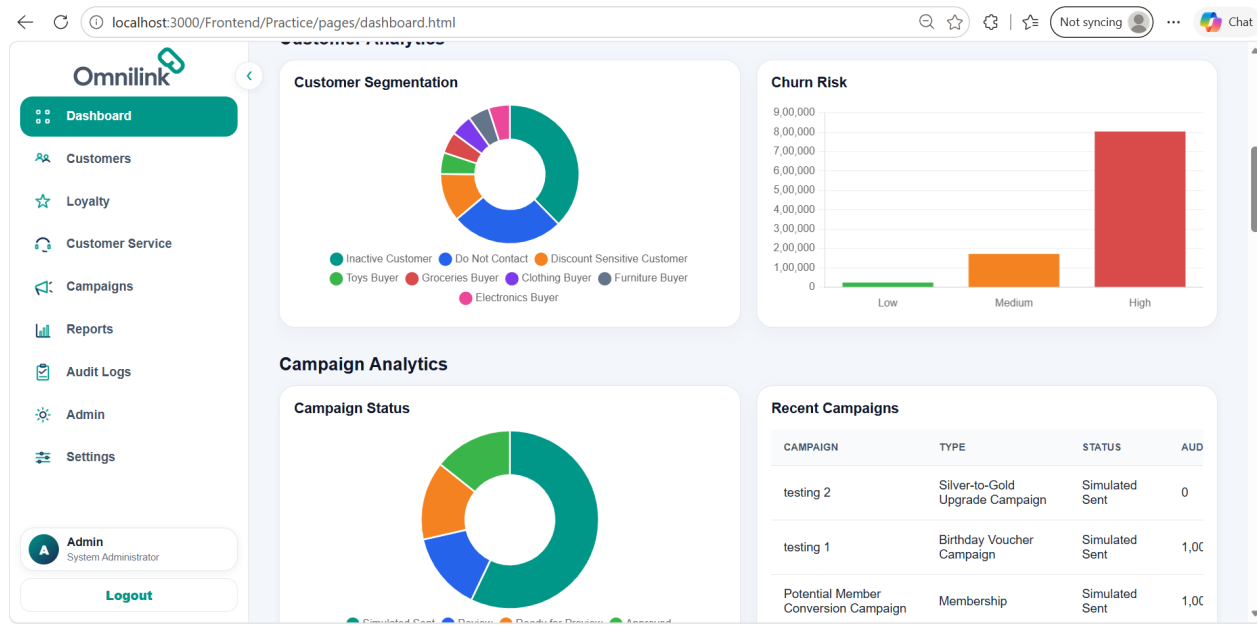


Figure 9.1: End-to-end project demonstration across the major CRM modules

5. Consent visibility.
6. Campaign creation and Discord simulation.
7. Customer Service ticket update.
8. Audit event verification.
9. Admin controls and data health.

9.4 Results

The final system demonstrates an integrated CRM workflow rather than isolated pages. Customer 360 data supports analytics, loyalty, consent, campaign targeting, and service context. Operational actions update the database and create history or audit records. Performance controls support a large customer dataset while retaining full-database summary metrics.

Conclusion and Future Work

10.1 Conclusion

OmniLink CRM demonstrates how customer data can be converted into coordinated business operations. The platform combines a central customer view, executive analytics, loyalty management, privacy-aware communication, campaign operations, service-ticket workflows, administration, reporting, and audit traceability.

The project also demonstrates full-stack integration. The frontend provides a consistent role-based interface. FastAPI provides modular and testable APIs. SQLite provides a portable database for the prototype. JWT, password hashing, consent rules, and audit logging provide security and governance. Caching, pagination, rule reuse, and optimized SQL improve responsiveness.

10.2 Future Work

- Replace SQLite with PostgreSQL or another enterprise database.
- Add background workers for large campaign execution.
- Integrate production Email, SMS, WhatsApp, and App providers.
- Use Microsoft Graph or a transactional email service for password recovery.
- Add refresh tokens, revocation, and centralized session management.
- Add automated unit, integration, API, and browser tests.
- Introduce machine-learning models for churn and propensity scoring.
- Normalize transactions, products, stores, and events into dedicated tables.
- Add centralized secrets management and HTTPS.

10.3 Final Project Contribution

Integrated CRM Contribution

OmniLink CRM connects customer truth, analytical intelligence, communication permission, loyalty decisions, campaign engagement, service operations, and governance around one customer foundation.

Bibliography

- [1] FastAPI Documentation, <https://fastapi.tiangolo.com/>.
- [2] SQLite Documentation, <https://www.sqlite.org/docs.html>.
- [3] Chart.js Documentation, <https://www.chartjs.org/docs/latest/>.
- [4] PyJWT Documentation, <https://pyjwt.readthedocs.io/>.